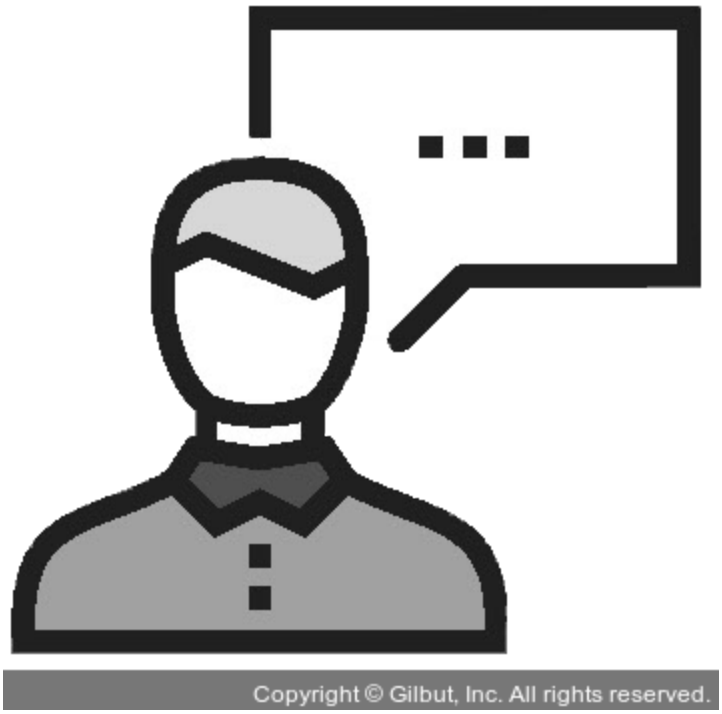
UNIT 13

숫자 데이터를 쉽게 다루게 돕는 numpy 라이브러리



⦿ 실습 데이터 Unit13/unit13-01.ipynb

지금까지 우리가 다룬 `matplotlib` 라이브러리는 아주 기초적인 내용이고, 그중에서도 자주 사용되는 기능들을 중심으로 살펴보았습니다. 더 자세한 내용이 궁금한 사람을 위해 이번 장에서는 matplotlib 홈페이지를 살펴보고 여러 가지 숫자 데이터를 다룰 수 있는 `numpy` 라이브러리를 실습해 봅시다.

1 matplotlib 홈페이지

matplotlib 홈페이지는 데이터 시각화와 관련된 다양한 예시를 제공하고 있습니다. 원하는 예시를 클릭하면 해당 그래프를 어떻게 그렸는지 알 수 있는 코드도 함께 제시하고 있습니다.

matplotlib.org에 접속한 후 상단에 있는 tutorials 메뉴를 클릭합니다.



그림 13-1 matplotlib 홈페이지

2 numpy 라이브러리 시작하기

먼저 간단한 함수부터 살펴보겠습니다. 다음 코드는 2의 제곱근()을 출력합니다.

```
import numpy
print(numpy.sqrt(2))
```

실행 결과

```
1.4142135623730951
```

그리고 앞에서 잠시 살펴봤던 것처럼 보통의 경우에는 다음과 같이 `numpy` 라이브러리를 `np`라는 이름의 별명으로 줄여서 쓸 수 있습니다.

```
import numpy as np
print(np.sqrt(2))
```

3 numpy 라이브러리를 활용해 그래프 그리기

0부터 5까지의 숫자가 랜덤으로 출력되는 실행 결과를 보며, 각 숫자가 출력되는 횟수를 쉽게 확인하는 방법을 고민할 수 있습니다. 히스토그램을 그려 각 숫자의 빈도가 한눈에 들어오도록 합시다. 그리고 `numpy` 라이브러리의 장점을 확인하기 위해 Unit 6에서 `random` 라이브러리와 리스트를 사용했던 코드와 비교해 보겠습니다.

numpy를 사용한 모드	Unit 6에서 사용한 코드

`import matplotlib.pyplot as plt`
`import numpy as np`
`dice = np.random.choice(6, 10)`
 	
`plt.hist(dice, bins =6)`
`plt.show()`
	`import matplotlib.pyplot as plt`
	`import random`
	`dice = []`
	`for i in range(10) :`
	`    dice.append(random.randint(1,6))`
	`plt.hist(dice, bins = 6)`
	`plt.show()`

표 13-1 numpy를 사용한 코드 vs. Unit 6에서 사용한 코드

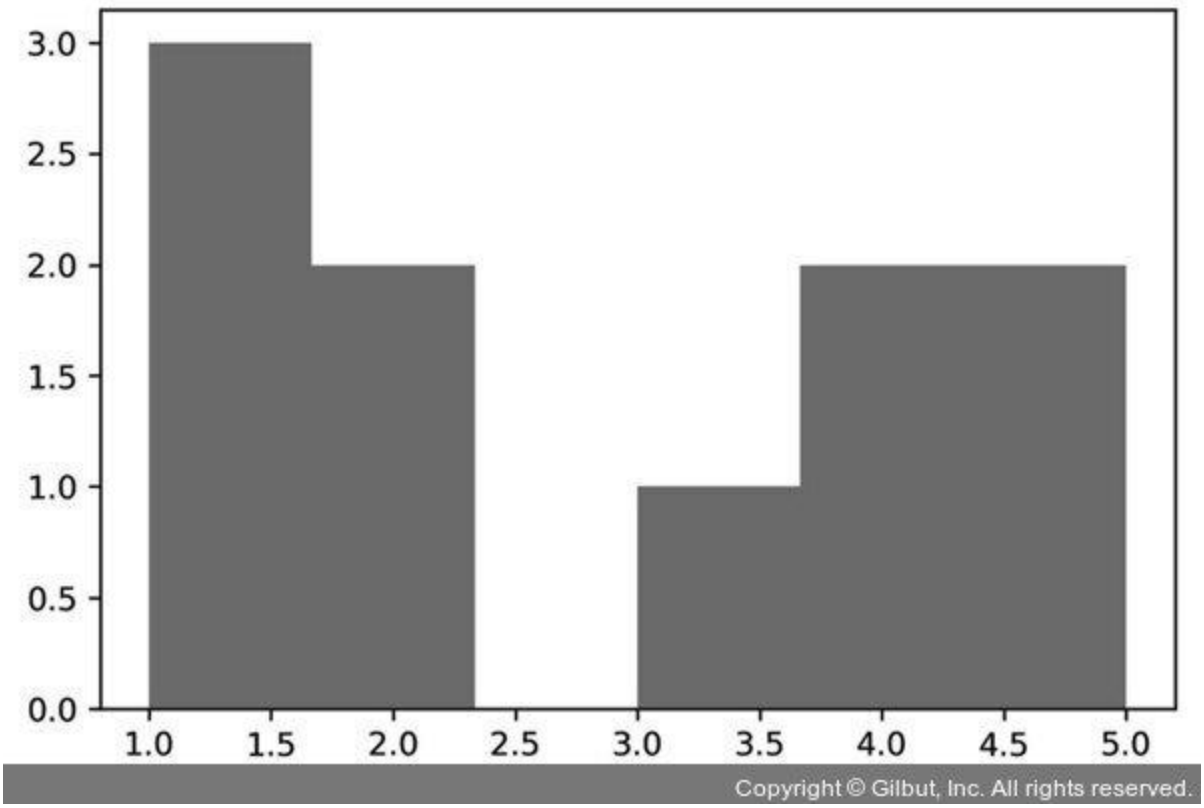


그림 13-5 두 가지 히스토그램 그리기 코드 실행 결과

4 numpy array 생성하기

`numpy` 라이브러리에서는 `ndarray`라는 특별한 데이터 타입의 배열이 사용됩니다. `numpy`를 사용하려면 `ndarray` 타입 배열을 다루는 데 익숙해질 필요가 있습니다. 'N차원 배열'이라는 의미를 지닌 `ndarry` 타입 배열을 지금부터는 배열 또는 `array`라고 부르겠습니다.

리스트와 같은 연속된 데이터는 다음과 같이 `numpy array`로 변환할 수 있습니다.

```
import numpy as np
a = np.array([1,2,3,4])
print(a)
```

실행 결과

```
[1 2 3 4]
```

5 numpy array의 다양한 활용

그런데 `np.zeros()`, `np.ones()` 함수는 있는데 왜 `np.twos()`, `np.threes()` 함수는 없을까요? 만약 초깃값이 5인 배열을 100개 만들고 싶으면 어떻게 하면 될까요? 바로 이 점이 `numpy array`를 우리가 알아야 하는 가장 큰 이유인 것 같습니다.

이해를 돕기 위해 다음 코드와 실행 결과를 살펴보겠습니다.

```
import numpy as np
a = np.zeros(10) + 5
print(a)
```

실행 결과

```
[5. 5. 5. 5. 5. 5. 5. 5. 5. 5.]
```